Assignment 3: Make It Visible

CS 356 UT Austin · Fall 2026 · Student handout

To run a network well, you need to know what it's doing: who's talking to whom, where traffic goes, what's struggling. But you can't rewind a network. All you'll ever know is what you recorded at the time. **The catch: you pick what to count, where, and how much to keep, before you know what you'll need.**

In A1 and A2 your program ran the network. In A3 we run it, and your switches just **watch**: they count packets, bytes, delays, whatever you think matters, and report what they see. Somewhere in the network we've hidden **eggs**: things that happen that you'll only catch if you were measuring the right thing in the right place.

Watching is not free, and that is the whole game. A switch's controller sees only a sample of events, never everything. Every report your switches send uses up part of a byte budget. Reports travel on the same links as customer traffic, but wait in their own small queue and only go when no customer packet is waiting. On a busy link your reports wait, and if their queue fills up, new ones get dropped. All your reports end up in one small log. That log is all you get back after a run, so make every byte count. Count in the data plane, or sample to the controller? Summarise at the switch, or ship raw events? Write on a timer, or only when something changes? Put your logging switch near the trouble, or in the middle? **So you're working on two things: how well you log, and how well you read what you logged.**

1 The hunt: truth runs and egg claims

You write two programs. `switch.wasm`, your instruments, runs on every switch. It doesn't forward anything (the network does that); it decides what to notice, count and report. It also picks **one logging switch**, which writes **your log** in a format you design. `detect.wasm`, your detector, reads your log after a run and answers **true or false**. Figure 1 shows how the two kinds of attempt fit together.

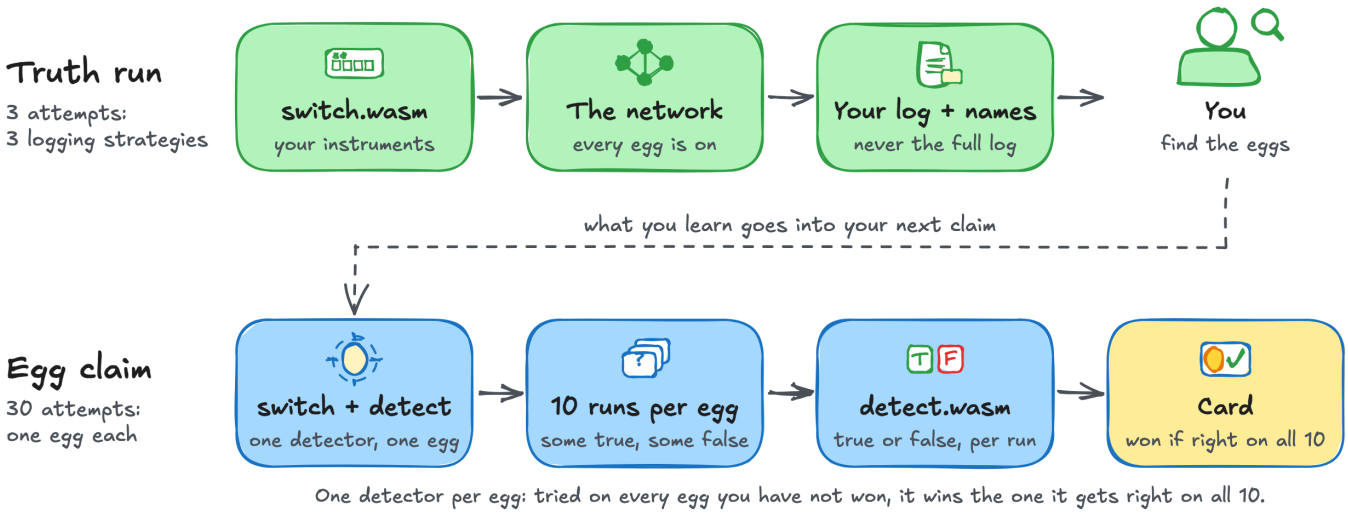


Figure 1: Truth runs and egg claims.

Truth runs (3 per team). Send us a `switch.wasm` and we'll run it on the hidden network. Your logging switch can write up to *[T]* KB of whatever you like. We send you back that log, plus a list of which address range belongs to which customer. That's it: you only see what your switches caught. Every egg shows up in all three runs, but each run has a new random seed and a heavier load, so timing and traffic shift. Look at what you got, then log better next time.

Egg claims (30 per team). A claim is your `switch.wasm` plus a `detect.wasm` built to catch **one egg**. You don't tell us which one. We test it against every egg you haven't won yet, and if your detector gets an egg right every time, you win it (Section 2 explains how we test). No logs come back, just a **card** listing the eggs you've won, by number, category and name. A won egg stays won, and sending the same claim twice won't help. Customer address ranges get reshuffled on every test, so recognise Netflix by how its traffic looks, not by where it sat in your truth runs.

2 Eggs

An egg is a **yes-or-no question about something that happened in a run**, like "Did a customer go quiet for a while?" or "Did someone scan another customer's addresses?". The answer is always a clear yes or no, and it's always about the network, not your program.

Finding them takes three skills. **Tell your customers apart.** Truth runs tell you which range belongs to which customer, but when we test your claims we shuffle the ranges, so you'll have to recognise each customer by its traffic. **Watch the physical network**: links and switches that go down, or stay up but work badly. **Read the traffic**: who sends what, to whom and when, including trouble nobody announces.

The eggs are hidden. There are 20, and we don't say what they are. Every egg happens in all three truth runs. Apart from the eggs, the truth runs have only normal customer traffic, with some randomness from run to run; we haven't planted anything else. And nothing your program does can make an egg happen or stop one.

How we test a claim. Every egg has its own 10 **claim runs**, on the same network as your truth runs, with every other egg switched on. The only thing that changes is the egg being tested: in some runs it happens (**true**), in the rest it doesn't (**false**) (Figure 2). A false run is either **off**, where the egg doesn't happen at all, or a **near miss**, where something close to it happens instead. Near misses are there to catch detectors that only check part of the egg.

Here's an example, the first practice egg below (the hidden graded eggs are different). Say the egg is "link 1–2 goes down and stays down." In an off run, no link goes down. In a near miss, a different link goes down, or 1–2 goes down and comes back. A detector that only checks "did some link go down?" says yes on that near miss, so it loses, like the second detector in Figure 2. One that checks "1–2 went down and stayed down" wins. And a detector that always says true loses too.

Three practice eggs. The kit has a **practice network** (not the hidden one) and the simulator, to run on your machine as often as you like. There you also get the **simulator trace** (the kit calls it the *full log*), a record of everything that happened in the run, so you can check what your switches actually caught. Its three eggs are not hidden, one per skill:

- **The broken link** (Failure). Partway through, link 1–2 goes down and stays down. *Off:* no link goes down. *Near misses:* another link goes down; 1–2 goes down and comes back.
- **The long pause** (Customer). Partway through, Spotify stops sending for at least 10 s, then carries on. *Off:* Spotify never pauses. *Near misses:* Spotify pauses for only 2 s; another customer pauses for 10 s. Ranges are reshuffled, so find Spotify first.
- **The scanner** (Traffic). Partway through, one host sends a single small packet to each of

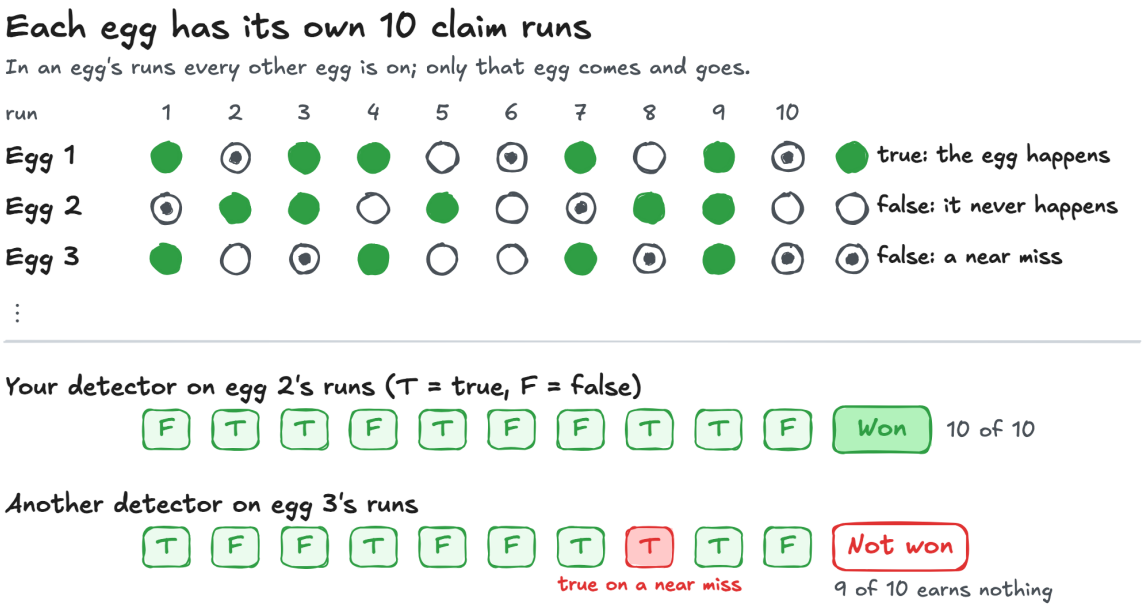


Figure 2: Each egg's claim runs, and two detectors.

at least 50 addresses in another customer's range, all within 5 s. *Off:* no host does. *Near misses:* one host sends many packets to just a few addresses; one host reaches 50 addresses, but spread over the whole run.

We've solved the broken link for you. The kit has a switch program and a detector that win it, so you can see how the whole thing works. The other two are yours. Each practice egg has **10 practice claim runs**, and the **practice checker** grades your claim on them just like the server does, showing you every answer (Figure 3). It's the only place to check that your detector says false when it should. Practice eggs earn no points.

3 Your log

Your log is the only thing your detector ever reads. Whatever an egg needs must get into it, arrive in time, and fit.

One switch writes it. You pick which one: the first switch to ask becomes the **logging switch** for the whole run. Only the logging switch can write to your log, so anything your other switches see has to be sent to it as reports. Each switch's controller sees at most 80 events a second, so what it reports is already a sample. These are simulated seconds, not real ones. The simulator keeps its own clock, so one simulated second can take more or less than a real second to run. A run lasts 40 simulated seconds, and every time in this handout is simulated time. Where you put the logging switch decides what reaches it, and how late. To help you choose, we'll publish a **map of the hidden network** with `SUBMITTING.md` when the server opens: every switch id and which switches are linked. It's the same in every run. Link speeds and delays aren't on it.

Two caps, on purpose. In a truth run your log can hold *[T]* KB. You don't know what the eggs are yet, so you need room to look around. In a claim it holds only *[S]* KB (we'll publish the number when the submission server opens). By then you know what your one egg needs, so log that and not much else. Explore wide, then cut down: that's the arc of the assignment. Logging

that fit fine in a truth run will overflow in a claim unless you trim it. A write that doesn't fit is lost completely.

It holds only what you wrote. Nothing is added for you: if you did not write the time, or which switch saw it, you do not have it. The format is yours to design, and `detect.wasm` must read it back. Your switch program can instrument for several eggs at once, but they all share this one log. Summarise, sample, and write only what your detector will read. The kit's logging SDK has the details.

For ideas on fitting a lot into a little space, these are good starting points (optional, suggestions only):

- sFlow: how switches sample packets instead of keeping them all.
- Reservoir sampling: a fair sample from a stream when you do not know how long it is.
- Count–min sketch and Misra–Gries: counting many things, and finding the biggest, in fixed memory.
- Monitoring Distributed Systems (Google SRE book): what is worth measuring.

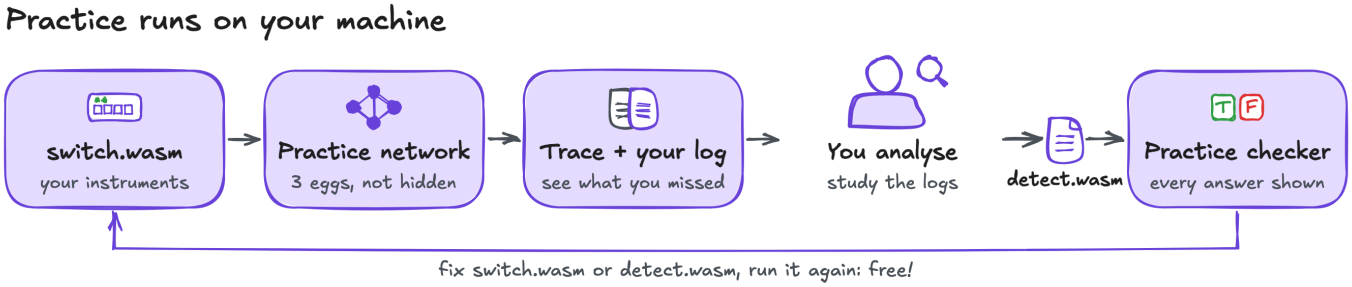


Figure 3: The practice loop.

4 Making your attempts count

You have two budgets, three truth runs and thirty claims. Spend them in that order: learn in the truth runs, then claim.

Truth runs: learn. They are the only time you see the hidden network, and there is no fourth. Plan each one before you spend it: what you want to learn, and what you will log to learn it. Debug your logging on the practice network first; a truth run spent finding a bug in your own code is wasted. Use the map to decide where your logging switch goes before your first run. Truth runs give you a bigger log ($\lfloor T \rfloor$ KB) to explore with; keep every one, because they are all your evidence.

Claims: commit. Each claim is a bet on one egg, the one its detector was built for. Each egg's 10 claim runs are fixed for the whole course: each has its own random seed, but you get the same 10 every time you claim. And the card only says what you won, never why you missed. So test the false side before you submit: would your rule still say yes if the same kind of thing happened somewhere else, at another time, or to another customer? Try it on the practice runs and against your truth logs. Then cut your logging down to what this one detector reads: the wide logging that served a truth run will not fit in a claim's log.

5 What you get

Everything from A2: the simulator and the Rust SDK. You won't need your A2 switch, since in A3 the network does all the forwarding. Start from the solved practice egg instead. New in this kit:

- **The practice network** and its **three practice eggs**, each with 10 practice claim runs and their answers. It is a different, smaller network from the one you are graded on, and you can run it as often as you like.
- **The simulator trace, locally.** On the practice network, every run gives you the trace and your log side by side, so you can check what your instruments caught against what really happened. `simlog_dump` prints a trace as text, and the **replay viewer** draws it.
- **A solved practice egg**: a switch program and a detector that win the broken link, as source, to start from.
- **The practice checker**, which runs your two programs on a practice egg's 10 runs and prints a card like the server's, with every run's answer shown. It also checks that both programs load, so a claim it accepts will not come back invalid.
- **The logging SDK**: how to name your logging switch, write to your log, send reports under the budget, and read your log back in `detect.wasm`.

6 Grading

The 20 eggs are worth **200 points** in all. Each egg is **easy**, **medium** or **hard**, and the harder it is, the more points it is worth. Your points are those of every egg you have won by the deadline; a won egg stays won.

150 points is 100% on A3. Below that, your grade is your points out of 150.

A4 bonus. Your A3 points also earn a bonus in A4, at one of three levels:

Level 1	100 points or more
Level 2	150 points or more
Level 3	200 points: every egg

What each level earns comes with the A4 handout.

7 Using AI

Use it however you like. AI knows the textbook ways to measure a network, and it'll write you a logging switch or a detector in a minute. What it's never seen is this network, or what your truth runs showed. Deciding what to watch, and what your detector should believe, is up to you, and the card won't tell you when you got it wrong.

8 Logistics

Server opens	Sunday, October 25, end of day, for truth runs and claims.
Due	Tuesday, November 3.
Teams	2 students.
Submitting	How to request a truth run and submit a claim, the log caps and the map of the hidden network are published in `SUBMITTING.md` when the submission server opens.